ToSh Cheat Sheet: Types, Modules, And Interop

User-defined types, CLR access, reflection, and native interop basics

README + examples + commands

Using And Require

```
using System.IO
using System.IO = IO
```

```
require "./utils.tosh"
require Inventory from "./toastlib.tosh"
require Inventory from "./toastlib.tosh" as Inv
require { Reporting, Utilities } from "./toastlib.tosh"
```

- `using` shortens CLR namespace references in the current lexical scope.
- `require` loads a ToSh script/module once per session and imports names lexically.
- Imported modules are accessed with dot notation, e.g. `Inv.sample_items()`.

Named Types

```
record Item(Name: string, Quantity: int)

enum StockState {
    Healthy, Low, Out
}
```

```
class Animal {
    prop Name: string? = null
    prop Sound: string? = null

    Animal(name: string, sound: string) {
        $this.Name = $name
        $this.Sound = $sound
    }

    func Describe() -> string {
        return $"The {$this.Name} says {$this.Sound}"
    }
}
```

Records are lightweight value objects; classes support constructors, methods, stored/computed properties, `static` members, and `shy` private members.

Modules

```
module Utilities {
    shy func hidden() { ... }

    export func banner(title: string) {
        writeline $"=== {$title} ==="
    }
}

Utilities.banner("Report")
```

CLR Interop

```
new System.Guid "d3b07384-d113-4ec6-a5d2-5b1d0d2e8c39"
new System.Random | call Next 1 100
call System.String Join ", " ["objects", "pipelines", "types"]
```

```
"hello".ToUpper()
String.Join(" + ", ["a", "b"])
Math.Round(Math.PI, 4)
```

Prefer fluent calls like `$obj.Method()` and `TypeName.Method()`; use `call` or `call-method` when the invocation has to stay dynamic or pipeline-shaped.

Command	Use
`new`	construct CLR or ToSh objects
`call`	invoke a CLR instance/static method
`call-method`	invoke a method by dynamic name
`cast`	convert values to a target type
`type-of`	return the runtime type

Reflection And Inspection

```
type-of $obj
describe-type list<int>
describe-type System.String
members System.DayOfWeek
methods $obj
constructors list<int>
```

```
get-props $obj
get-prop $obj Name
set-prop $obj Name "Toast"
del-prop $obj Name
has-prop $obj Name
has-method $obj ToString
```

```
inspect $obj
inspect --flat
name-of $obj
```

Dynamic records and many runtime objects expose a shell-record interface, so reflection commands work well across both CLR objects and ToSh-native shapes.

Assembly Loading

```
load-assembly ./MyLib.dll
describe-type MyCompany.Tools.Widget
constructors MyCompany.Tools.Widget
```

Load an assembly when a CLR type is not already visible to the session; after that, `new`, `cast`, and dot-syntax calls work normally.

Pattern For Domain Code

```
module Inventory {
    record Item(Name: string, Quantity: int)
    enum StockState { Healthy, Low, Out }

    func state_of(item: Item) -> StockState {
        return match ($item.Quantity) {
            _ <= 0 => StockState.Out
            _ < 3 => StockState.Low
            default => StockState.Healthy
        }
    }
}
```

```
var bread = new Inventory.Item("Bread", 2)
Inventory.state_of($bread)
```

Native Interop Basics

```
require native "./libcrypto.so" as Crypto

bind Crypto {
    func sha256(data: byte-ptr, len: ulong) -> byte-ptr
}
```

```
alloc buffer = 1024
native-write $buffer "hello"
native-read $buffer
size-of int
offset-of MyStruct FieldName
native-free $buffer
```

Native tool	Purpose
`alloc` / `native-free`	manage unmanaged buffers
`native-read` / `native-write`	copy values to and from native memory
`size-of` / `offset-of`	inspect unmanaged layouts

Use records for compact data, classes for behavior, CLR interop for existing .NET types, and native interop only when managed APIs are not enough. ToSh lives directly on top of the CLR, so shell pipelines, user-defined types, and .NET APIs can coexist in the same script.

ToSh Cheat Sheet: Types, Modules, And Interop

Visibility, member forms, reflection workflow, and safer native patterns

Visibility And Scope

```
module Utils {
    shy var counter = 0
    export func get-count() => $counter
    global var DEBUG = false
}
```

Modifier	Effect
`shy`	private to the current lexical scope or type
`export`	visible when a module is imported
`global`	session-wide binding outside local scope

Class Member Forms

```
class HexColor {
    prop Hex: string? {
        get => $this.hex_value
        set => $this.hex_value = $value
    }

    shy prop hex_value: string? = null
    static func Green() => new HexColor("#00FF00")
}
```

- Stored property: `prop Name:  Type = value`
- Computed property: `prop Magnitude:  double => (...)`
- Getter/setter property: use a block with `get` and `set`
- Static members hang off the type: `HexColor.Green()`

Reflection Workflow

```
types System.String
describe-type list<int>
members DateTimeOffset
methods "toast"
constructors System.Random
```

Assembly Loading

```
load-assembly ./MyLib.dll
types Widget
describe-type MyCompany.Tools.Widget
constructors MyCompany.Tools.Widget
```

Object Mutation Tools

```
var original = {| Name = "Bread", Qty = 2 |}
var copy = (clone $original)
set-prop $copy Qty 5 | ignore
get-prop $copy Qty
del-prop $copy Qty
```

```
has-prop $copy Name
has-method "hello" ToUpper
get-props $copy
get-methods "toast"
```

Dynamic records are great for ad-hoc pipeline shapes; records and classes are better when you want named, reusable domain types.

Module Import Patterns

```
require "./utils.tosh"
require Inventory from "./toastlib.tosh"
require Inventory from "./toastlib.tosh" as Inv
require { Reporting, Utilities } from "./toastlib.tosh"
```

- `require` is lexical and cached once per session.
- `using` only aliases CLR namespaces; it does not run scripts.
- Use module qualification when you want APIs to stay discoverable in larger scripts.

CLR Call Patterns

```
new System.Random().Next(1, 100)
"hello".Replace("h", "y")
String.Join(", ", ["a", "b", "c"])
Math.Round(Math.PI, 3)
```

```
call System.String Join ", " ["x", "y"]
call-method "hello" Replace "llo" "y"
```

Choose The Surface

Tool	Prefer it when
`record`	you want immutable value-shaped data
`class`	you need behavior, methods, or private state
`module`	you are grouping functions and types into an API
`CLR type`	.NET already has the exact thing you need
`native ABI`	managed APIs are not enough

Native Signature Patterns

```
bind native "libc.so.6" as LibC {
    func abs(int) -> int callconv cdecl
    func strlen(string) -> nuint
    func memcpy(nint, nint, nuint) -> nint
}
```

```
require native "./libcrypto.so" as Crypto
bind Crypto {
    func sha256(data: byte-ptr, len: ulong) -> byte-ptr
}
```

Native type	Typical use
`nint / nuint`	pointer-sized ints
`byte-ptr`	raw buffer pointers
`cstring`	C string interop surfaces
`out / ref`	by-reference native parameters

Buffer Workflow

```
var src = (native-alloc 6)
var dst = (native-alloc 6)
native-write $src "toast"
LibC.memcpy($dst, $src, 6)
native-read cstring $dst
native-free $src $dst
```

```
alloc counter = long
native-write $counter 42
native-read long $counter
native-free $counter
```

Layout Inspection

```
size-of int
size-of nint
offset-of MyStruct Field1
```

Native Safety Rules

- Prefer `cstring` or pointer types for native string surfaces.
- Free every buffer allocated with `native-alloc` / `alloc`.
- Keep native signatures explicit: pointer types, by-ref mode, and calling convention.

Rule of thumb: stay in CLR land unless you truly need a native ABI. When you do cross that boundary, spell signatures explicitly and free every buffer you allocate.